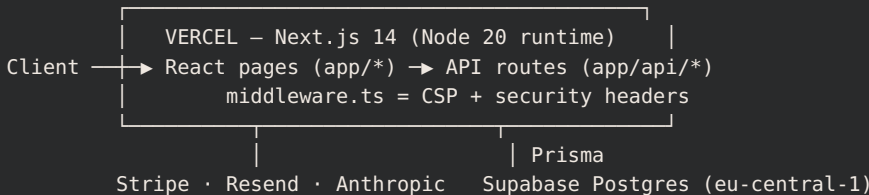


BirdNest — Technical & Deployment Guide

For: Technology / Development Department · Date: 22 June 2026
Repo: moatazsamir0044-jpg/first-project (default branch `main`)
Purpose: How the codebase is built, what it runs on, and exactly what to do to go live for bookings & payments.

1 · System Overview

BirdNest is a **server-rendered Next.js 14 (App Router)** application for holiday rentals. Front-end pages and the backend API live in one Next.js project, deployed together to **Vercel**. Data is in **PostgreSQL on Supabase**, accessed via **Prisma**. Payments via **Stripe**, email via **Resend**, an optional chatbot via **Anthropic**.



2 · Tech Stack (what it is built on)

Layer	Technology	Version	Notes
Framework	Next.js (App Router)	14.2.35	Pages + API in one project
Language	TypeScript (strict)	^6	Build fails on type errors (intentional)
UI / Styling	React / Tailwind CSS	^18 / ^3.4	Brand tokens in <code>tailwind.config.ts</code>
ORM	Prisma	^5.22	Schema in <code>prisma/schema.prisma</code>
Database	PostgreSQL (Supabase)	PG 17	<code>birdnest-production</code> , eu-central-1
Auth	NextAuth / Auth.js	5.0.0-beta.31	Credentials + Google · JWT sessions
Payments	Stripe	srv ^22 / js ^9	API version <code>2026-05-27.dahlia</code>
Email	Resend	^6.12	<code>lib/email.ts</code>
i18n	next-intl	^4.13	EN / AR · <code>messages/</code>
AI chatbot	@anthropic-ai/sdk	^0.104	<code>app/api/chat/*</code>
Maps / Validation	Mapbox GL / Zod	^3 / ^4	All API inputs Zod-validated
Hosting	Vercel	—	Project <code>first-project</code> , region <code>iad1</code>

⚠ **Lockfile split:** `vercel.json` uses `pnpm` but the repo ships both `pnpm-lock.yaml` and `package-lock.json`. Pick one package manager and delete the other lockfile to keep installs deterministic.

3 · Repository Layout

app/	App Router – api/ (backend), page routes, ar/ (Arabic), layout.tsx, sitemap.ts, robots.ts
components/	React components by feature (auth, booking, home, listing-detail, listings, layout, shared, blog)
lib/	auth.ts · prisma.ts · stripe.ts · password.ts (scrypt) rate-limit.ts · email.ts · policies.ts · formatters.ts countries.ts · mock-data.ts ◀ demo data pages still render
prisma/	schema.prisma · migrations/ (3) · seed.ts (12 listings)
middleware.ts	CSP nonce + security headers
content/ messages/	MDX/blog · i18n strings

4 · Data Model (8 Prisma models)

All 8 tables created in Postgres (eu-central-1), RLS-enabled, **12 listings + 36 reviews seeded**. Prices stored as **integers (EGP)**; Stripe charges in piastres (`total × 100` , currency `egp`).

Model	Key fields / purpose
User	email, <code>passwordHash</code> , role (default "user"); relations to accounts/sessions/bookings
Account / Session / VerificationToken	Standard NextAuth adapter tables
Listing	slug, title(Ar), pricePerNight, utilitiesEst, cleaningFee, beds/baths/maxGuests, amenities[], images[], rating, eligibility, lat/lng. Indexed: area, isActive, price, rating
Booking	reference(unique), listingId, userId?, guest{Name,Email,Phone}, <code>nationality</code> , dates, totalPrice, status (pending/confirmed/cancelled), <code>stripePaymentId</code> , eligibilityType
Review	listingId, author, rating, comment, source
ContactSubmission	name, email, enquiryType, message

5 · API Surface (app/api/*)

All handlers Zod-validate input, are rate-limited, and use Prisma. **These exist and work against the live DB** — the gap is the front-end not calling all of them yet (§9).

Route	Method	Purpose
/api/listings	GET	List/filter active listings
/api/listings/[slug]	GET	Single listing
/api/listings/[slug]/reviews · /availability · /view	GET/ POST	Reviews · date availability · view counter
/api/search · /api/amenities	GET	Search with filters · amenity list
/api/bookings	POST/ GET	Create booking · look up by email/reference
/api/payments/create-intent	POST	Create Stripe PaymentIntent → clientSecret + breakdown
/api/webhooks/stripe	POST	

Route	Method	Purpose
		payment_intent.succeeded/failed, charge.refunded → status + email
/api/auth/register · /[...]nextauth]	POST/*	Register (policy + rate limit) · NextAuth
/api/contact · /api/reviews · /api/chat/*	POST	Contact form · review submit · Anthropic chatbot
/api/admin/*	*	Dashboard, bookings, listings CRUD (guarded by ADMIN_SECRET_KEY)

6 · Authentication (lib/auth.ts)

- NextAuth v5. **Credentials (email/password) + Google OAuth**. Session strategy = **JWT**; **role** added to session.
- `trustHost: true`; secret = `NEXTAUTH_SECRET ?? AUTH_SECRET`.
- Passwords hashed with Node's built-in **script** (no native deps — serverless-friendly), constant-time verify.
- Register endpoint: 8+ chars upper/lower/number, rate-limited, enumeration-safe errors.

7 · Payments — Current vs Target

Backend payment path is fully built. The checkout UI is NOT wired to it — it simulates payment.

Current in `components/booking/QuickCheckout.tsx` — card fields are plain inputs, not Stripe Elements:

```
// Simulated until live keys are wired.
await new Promise(r => setTimeout(r, 1500)) // ◀ FAKE: no charge, no booking
onComplete(form)
```

Target flow to implement:

1. `POST /api/payments/create-intent` → receive `clientSecret`.
2. Mount **Stripe Elements** (`@stripe/react-stripe-js`) with the `clientSecret`; replace plain card inputs with `<PaymentElement>`.
3. `stripe.confirmPayment(...)`.
4. `POST /api/bookings` with `stripePaymentId` (status starts `pending`).
5. Stripe fires `payment_intent.succeeded` → webhook flips booking to `confirmed` + sends email.

Steps 1, 4, 5 already exist server-side. The remaining work is steps 2–3 plus two fetch calls.

8 · Security (already in place)

- `middleware.ts`: per-request **nonce CSP** + HSTS (2y, preload), `X-Frame-Options: DENY`, `nosniff`, Referrer-Policy, locked Permissions-Policy. CSP allowlists Stripe/Mapbox/Google/Unsplash/Vercel.
- `poweredByHeader: false`; all inputs Zod-validated; rate limits on register/booking/payment.
- Stripe webhook verifies signature against `STRIPE_WEBHOOK_SECRET`.
- Supabase RLS enabled on all tables; Prisma connects as owner (bypasses RLS by design — anon API locked).

9 · Gaps Blocking Go-Live (with exact files)

P0 Front-end renders mock data, not the DB

These import the mock *arrays* instead of the API/Prisma — convert to server components querying Prisma, or fetch the existing routes (importing the `Listing` type is fine):

```
app/listings/page.tsx          app/book/[slug]/page.tsx
app/listings/[slug]/page.tsx  app/booking/page.tsx
  (generateStaticParams = mock) app/compare/page.tsx
app/wishlist/page.tsx          app/sitemap.ts
components/home/FeaturedListings.tsx · components/shared/CompareBar.tsx
```

P0 Checkout simulates payment

`QuickCheckout.tsx` — implement the \$7 target flow (Stripe Elements + two fetches).

P0 Vercel env vars not set

See §10 — nothing runtime works without these.

P1 Prisma migration drift

- `Booking.nationality` is in the schema + live DB but **no migration creates it** — add a migration so a fresh `migrate deploy` is correct.
- Live DB was built from the schema directly; its `_prisma_migrations` doesn't list the 3 repo migrations. **Baseline it:** `prisma migrate resolve --applied <name>` for each.

P1 In-memory rate limiter

`lib/rate-limit.ts` uses a process-local Map — ineffective across serverless instances. Move to **Upstash/Redis**.

P2 Pre-launch & housekeeping

- Independent security review / pen-test; connect domain + DNS; verify Resend deliverability; native review of Arabic strings.
- Duplicate lockfiles — keep one. `vercel.json` region `iad1` (US) vs DB in EU — consider moving Vercel to an EU region (e.g. `fra1`) for latency.

10 · Environment Variables (set in Vercel → Production, then Redeploy)

The Vercel MCP integration is read-only for config — set these in the **Vercel dashboard** or via `vercel env`. Never commit them.

Required to function

Var	Source / notes
DATABASE_URL	Supabase → Connect → Transaction pooler (port 6543); replace [YOUR-PASSWORD]. Pooler for runtime, direct (5432) only for migrations.
NEXTAUTH_SECRET	<code>openssl rand -base64 32</code> — fresh, don't commit
NEXTAUTH_URL	Production domain (e.g. <code>https://birdnestlife.com</code>)
NEXT_PUBLIC_SITE_URL	Same production domain

Required for payments

Var	Notes
STRIPE_SECRET_KEY	Stripe live key
STRIPE_PUBLISHABLE_KEY	Used client-side by Stripe Elements
STRIPE_WEBHOOK_SECRET	After adding endpoint <code>.../api/webhooks/stripe</code> in Stripe

Optional: `GOOGLE_CLIENT_ID/SECRET` (login) · `RESEND_API_KEY` (email) · `ANTHROPIC_API_KEY` (chatbot) · `NEXT_PUBLIC_MAPBOX_TOKEN` · `NEXT_PUBLIC_WHATSAPP_NUMBER` · `ADMIN_SECRET_KEY` , `ADMIN_EMAIL` .

```
next.config.mjs serverActions.allowedOrigins lists birdnestlife.com — confirm that's the real production domain.
```

11 · Deployment Runbook (to first live booking)

1. Wire pages to the DB (P0 #1) — verify seeded listings render on `/listings` .
2. Wire Stripe into checkout (P0 #2).
3. Add `nationality` migration; `migrate resolve --applied` the 3 existing migrations against the live DB.
4. Set env vars in Vercel (§10) for Production.
5. Add the Stripe webhook endpoint → copy `STRIPE_WEBHOOK_SECRET` into Vercel.
6. Redeploy on Vercel.
7. **E2E test on the prod URL:** browse → listings appear; book with a Stripe **test** card → intent created → booking row written → webhook confirms → email received → shows in `/account` .
8. Switch Stripe to **live** keys, connect domain + DNS, re-test once.
9. Schedule the security review before announcing.

```
npm install && npm run build # prisma generate + next build (type+lint errors FAIL build)
npx tsc --noEmit · npx next lint # both should be clean
```

12 · What the Dev Team Should Start Doing Now

This week (unblocks go-live):

1. Assign one owner to **DB-wiring** (P0 #1) and one to **Stripe checkout** (P0 #2) — independent, run in parallel.
2. Vercel owner: set **env vars** (§10) + add the **Stripe webhook** (≈1 hour; blocks all runtime testing).
3. Add the **nationality migration** and **baseline the live DB**.

Immediately, as standing practice:

1. Resolve the **lockfile / package-manager** split.
2. Stand up a **Vercel Preview/staging** env with its own Supabase project/branch — never test against prod data.
3. Move the **rate limiter** to **Upstash**.
4. Add **tests around the booking + payment path** and wire CI on PRs.
5. Keep the **branch** → **PR** → **squash-merge** workflow off `main` .

Before public launch:

1. Independent **security review / pen-test**.

- 2. Verify **Resend** deliverability (SPF/DKIM on sending domain).
- 3. Native-speaker review of **Arabic** content.
- 4. Build a minimal **admin UI** on the existing `/api/admin/*` routes for non-developers.

Bottom line

The engine is built: data model, full API, auth, security, and a seeded production database all exist and work. What's left is **wiring the front-end to the database**, **connecting the checkout to the existing Stripe backend**, and **setting the Vercel environment variables** — plus a migration baseline. These are well-scoped integration tasks, not new systems. With focused effort the site can take its first real, paid booking within roughly **one week**.

13 · Key Identifiers

Thing	Value
GitHub repo	moatazsamir0044-jpg/first-project (default main)
Vercel project	first-project
Supabase project	birdnest-production · ref ngeqlmbqgauhghmouq · eu-central-1 · PG 17
Supabase API URL	https://ngeqlmbqgauhghmouq.supabase.co
⚠ Do NOT touch	Supabase trnvibgjjcahegjjoaz – different app (task tracker)